

WinInstaller Pro User Guide

Author packages, export builds, and run safe per-user installations

WinInstaller Pro

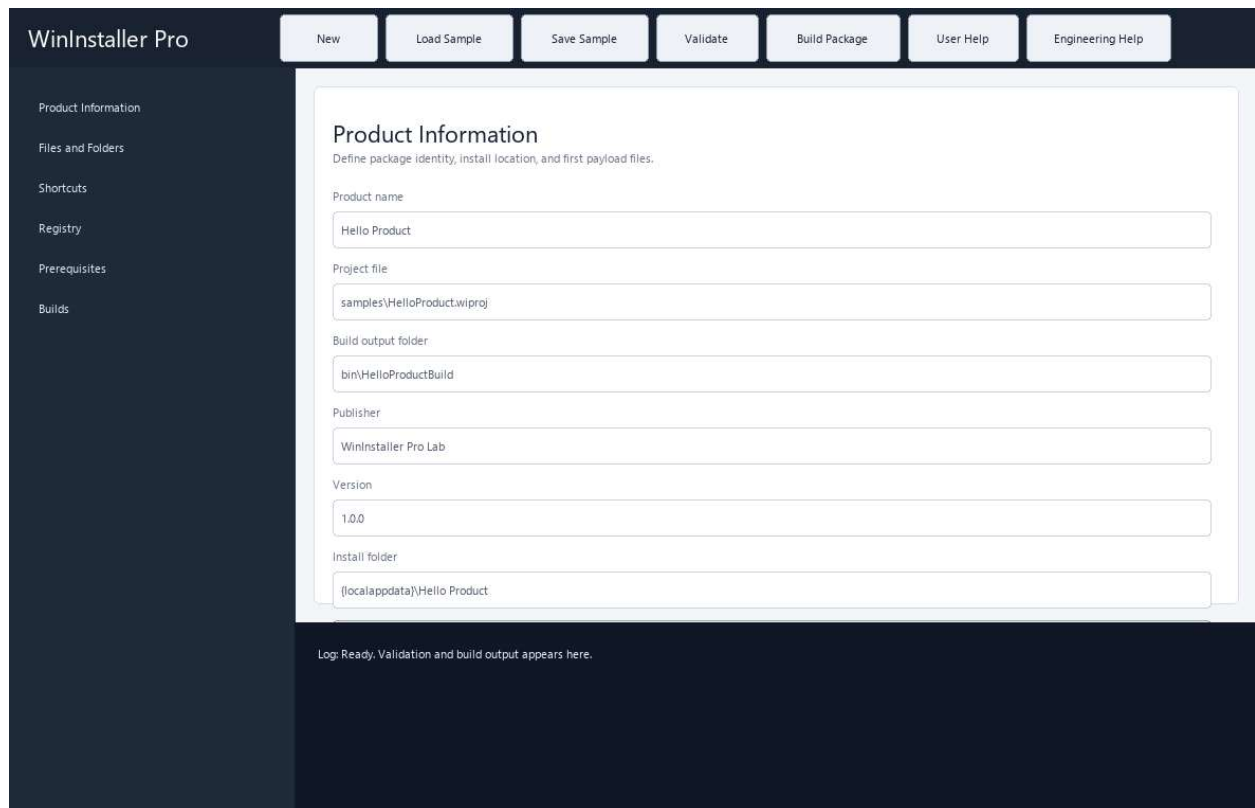


Figure 1. WinInstaller Pro authoring screen with visible designer-backed controls.

Purpose

WinInstaller Pro creates per-user Windows installer packages from Delphi FMX project data. It supports payload files, shortcuts, HKCU registry entries, per-user environment variables, file associations, prerequisites, feature metadata, license text, signing metadata, repair, verify, uninstall, and export archives.

Note: The current product intentionally avoids HKLM writes, services, drivers, arbitrary custom actions, and elevated machine-wide installs.

Main Workflow

1. Open WinInstallerPro.exe from the bin folder.
2. Load the sample project or enter a project file path.
3. Fill in product identity, install folder, payload files, shortcuts, registry entries, environment variables, file associations, prerequisites, features, license text, and signing metadata.

4. Click Validate.
5. Click Build Package.
6. Distribute the generated export folder or ZIP archive.

Authoring Fields

Area	What to enter	Notes
Project file	Path to the .wiproj file	Saved as JSON.
Build output folder	Folder where the export is written	Contains package EXE, runtime, payload, reports, and ZIP.
Product identity	Product, publisher, version	Used in reports, setup wizard, and uninstall registration.
Install folder	Target folder, including {pf} or {localappdata}	Protected Windows/system folders are blocked.
Payload files	Files to install	Duplicate target names are blocked.
Shortcuts	Location Name Target	Locations: InstallFolder, Desktop, StartMenu.
Registry	HKCU\Key Name Value	HKLM is intentionally blocked.
Environment	Name Value	Written under HKCU Environment.
File associations	Extension Description Target	Per-user association under HKCU Software\Classes.
Prerequisites	Name Mode Source	Recorded and reported as metadata.
Features	Name State Description	Recorded and shown in build/setup summaries.

Build Output

File or Folder	Purpose
<Product> Package.exe	FMX setup wizard for the exported package.
install.wiproj	Packaged project manifest.
WipRuntime.exe	Console runtime for automation.
payload\	Staged files to install.
build-report.txt	Build inventory.
safety-report.txt	Safety guard summary.
<Product> Export.zip	ZIP archive of the package export.

Setup Wizard

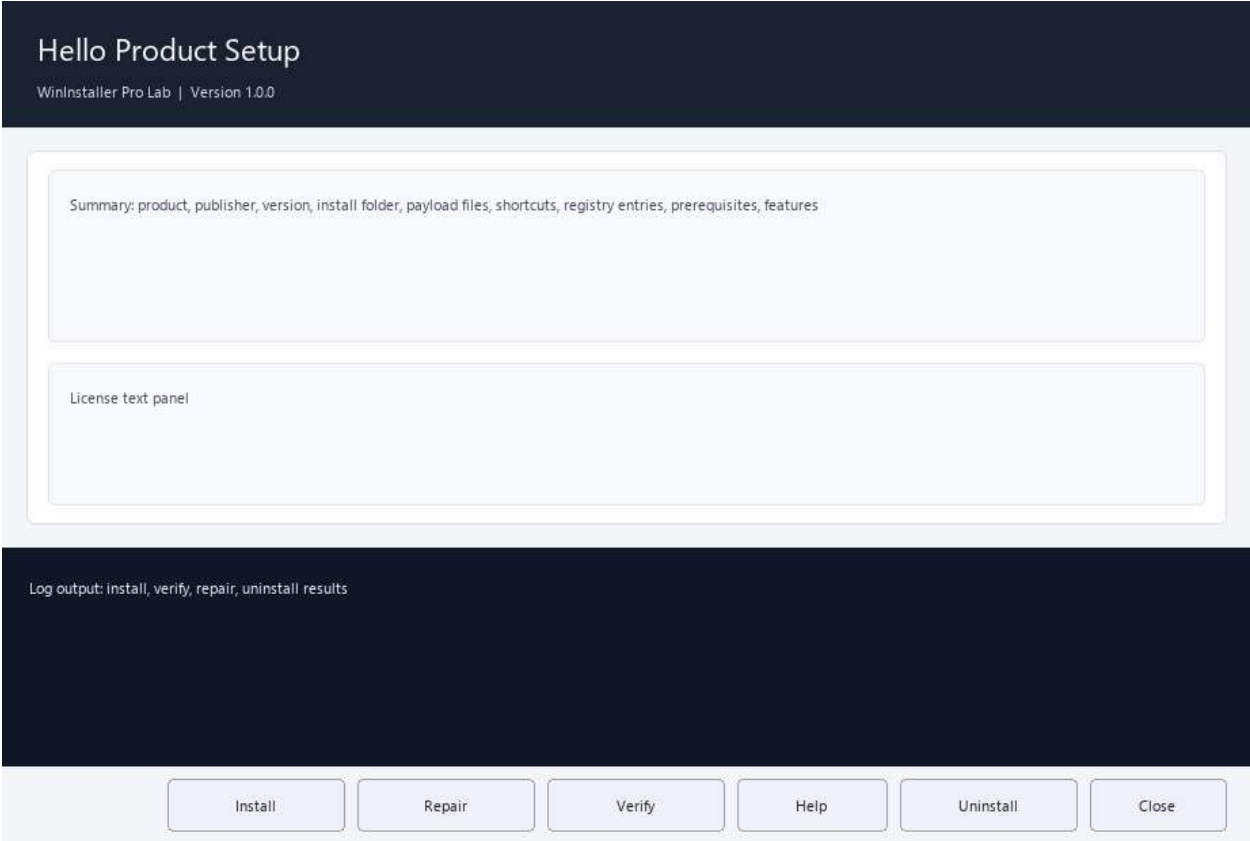


Figure 2. Setup wizard screen with Install, Repair, Verify, Help, Uninstall, and Close actions.

Action	Result
Install	Copies payload files, creates shortcuts, writes HKCU registry/environment/file association entries, and writes the uninstall manifest.
Verify	Reads the installed manifest and checks installed file hashes without changing the system.
Repair	Re-copies payload files and refreshes per-user entries while preserving the original uninstall manifest.
Uninstall	Uses the manifest to remove owned files and restore safe backups where appropriate.
Help	Opens UserHelp.chm beside the executable.

Command-Line Runtime

The console runtime supports these commands:

```
WipRuntime.exe install.wiproj --install
```

```
WipRuntime.exe install.wiproj --verify
```

```
WipRuntime.exe install.wiproj --repair
```

```
WipRuntime.exe install.wiproj --uninstall
```

Safety Expectations

- Do not use protected Windows/system folders as install targets.
- Use HKCU registry paths only.
- Do not expect driver, service, or HKLM support in this safe build.
- Review build-report.txt and safety-report.txt before distributing a package.
- Keep UserHelp.chm beside the runtime EXE when Help access is needed.

Creating a Build: Complete Step-by-Step Instructions

This section explains how to create a WinInstaller Pro build from start to finish. It covers the required project information, optional package information, validation, export, and the files produced by the build process.

Before you begin: Make sure the files you plan to install already exist on disk. WinInstaller Pro validates payload paths before it creates an export build.

Step 1. Open the Authoring Application

Run the authoring application from the project output folder:

C:\New Delphi Projects\WinInstaller Pro\bin\WinInstallerPro.exe

The authoring screen is where you define the installer project, validate it, and create the export build.

Step 2. Choose or Create a Project File

In the Project file field, enter the full path to the .wiproj file that stores the installer definition. This file is saved as JSON and contains the package identity, install rules, payload list, shortcuts, registry entries, environment variables, file associations, prerequisites, features, license text, and signing metadata.

Field	Required	Example	Purpose
Project file	Yes	C:\New Delphi Projects\WinInstaller Pro\samples\HelloProduct.wiproj	Stores the installer project definition.
Build output folder	Yes	C:\New Delphi Projects\WinInstaller Pro\bin\HelloProductBuild	Receives the exported package files.

To begin from the included sample, click Load Sample. To create a new project definition, click New, fill in the fields, and then click Save Sample or build the package.

Step 3. Enter Required Product Information

The following fields are required for a valid build:

Required field	What to enter	Validation rule
Product name	The display name of the product being installed.	Cannot be blank.
Publisher	The company, author, or organization name.	Recommended for reports and uninstall registration.

Version	The product version, such as 1.0.0.	Cannot be blank.
Install folder	The target folder where files will be installed.	Cannot be blank and cannot be a protected Windows/system folder.
Project file	The .wiproj path to save.	Must be writable.
Build output folder	The export destination folder.	Must be writable.

Install folder safety: Protected folders such as C:\Windows, C:\Windows\System32, and drive roots are blocked. Use a per-user or application-specific folder whenever possible.

Supported folder token examples:

Token or path	Meaning
{localappdata}\ProductName	Installs under the current user LocalAppData folder.
{pf}\ProductName	Expands to Program Files, but may require future elevated support depending on permissions.
C:\Some\Application\Folder	Uses an explicit folder path.

Step 4. Add Required Payload Files

Payload files are the files that will be copied into the install folder. At least one payload file is normally expected for a useful installer.

1. Enter a full path in Payload file path.
2. Click Add Payload File.
3. Confirm the file appears in the Payload files list.
4. Repeat for each file that should be installed.
5. To remove a file, select it in the list and click Remove Selected File.

Payload filename collisions: Two payload files with the same final filename are blocked because they would overwrite each other in the install folder.

Step 5. Add Optional Shortcuts

Shortcuts are optional. They can be created in the install folder, on the Desktop, or in the Start Menu.

Shortcut field	Required for shortcut	Example
Shortcut location	Yes	InstallFolder, Desktop, or StartMenu
Shortcut name	Yes	Hello Product
Shortcut target	Yes	hello.txt or MyApp.exe

1. Enter the shortcut location. Use InstallFolder if you want the shortcut stored under the install folder.
2. Enter the shortcut display name.
3. Enter the installed target file name or full target path.
4. Click Add Shortcut.
5. Confirm the shortcut appears in the Shortcuts list.

Step 6. Add Optional HKCU Registry Entries

Registry entries are optional and are limited to HKCU for safety. HKLM and machine-wide registry writes are intentionally blocked in this safe build.

Registry field	Required for registry entry	Example
Registry key	Yes	HKCU\Software\MyCompany\MyProduct
Registry value name	Yes	InstallFolder
Registry value data	Optional	C:\Users\User\AppData\Local\MyProduct

1. Enter an HKCU registry key.
2. Enter the value name.
3. Enter the value data.
4. Click Add Registry Entry.
5. Confirm the entry appears in the Registry entries list.

Registry safeguard: During uninstall, WinInstaller Pro removes or restores registry values according to the install manifest and skips values that were changed after installation.

Step 7. Add Optional Environment Variables

Environment variables are optional and are written under the current user environment scope.

Environment field	Required	Example
Name	Yes	HELLO_PRODUCT_HOME
Value	Yes	C:\Users\User\AppData\Local\Hello Product

1. Enter the environment variable name.
2. Enter the value.
3. Click Add Environment Variable.
4. Confirm the variable appears in the Environment variables list.

Step 8. Add Optional File Associations

File associations are optional and are written under HKCU\Software\Classes for the current user.

Association field	Required	Example
File extension	Yes	.hello
File type description	Yes	Hello Product Document
Association target	Yes	hello.txt or MyApp.exe

1. Enter the extension, including the leading dot if desired.
2. Enter a readable file type description.
3. Enter the installed target file name or full path.
4. Click Add File Association.
5. Confirm the association appears in the File associations list.

Association safeguard: If another association existed before install, the uninstall process restores it when safe to do so.

Step 9. Add Optional Prerequisites

Prerequisites are optional metadata in the current build. They are recorded in the project and reports so the package author can document required external runtimes or installers.

Prerequisite field	Required	Example
--------------------	----------	---------

Prerequisite name	Yes	.NET Desktop Runtime
Prerequisite mode	Yes	Manual
Prerequisite source	Optional	https://dotnet.microsoft.com/

1. Enter the prerequisite name.
2. Enter the mode. Use Manual for requirements that must be handled outside the installer.
3. Enter a URL or local source path if available.
4. Click Add Prerequisite.
5. Confirm the prerequisite appears in the Prerequisites list.

Step 10. Add Optional Feature Metadata

Features are optional metadata that describe product components. In the current build, they appear in summaries and reports.

Feature field	Required	Example
Feature name	Yes	Core
Feature state	Yes	Required or Optional
Feature description	Optional	Core application files

1. Enter the feature name.
2. Enter Required or Optional as the feature state.
3. Enter a description if useful.
4. Click Add Feature.
5. Confirm the feature appears in the Features list.

Step 11. Add Optional License Text

License text is optional. If entered, it is packaged into install.wipproj and shown in the setup wizard license panel.

Step 12. Configure Optional Signing Metadata

Digital signing is optional. Enable it only when you have a valid signing certificate and SignTool available.

Signing field	Required when signing is enabled	Example
---------------	----------------------------------	---------

Digitally sign exported package	Yes	Checked
SignTool path	Yes	signtool.exe
Signing certificate path	Yes	C:\Certificates\CodeSigning.pfx
Timestamp URL	Optional	http://timestamp.digicert.com

Signing validation: If signing is enabled, the certificate path must exist and SignTool must be available, or the build fails validation/signing.

Step 13. Validate the Project

Click Validate before creating a build. Review the log panel for validation messages. Resolve any reported errors before continuing.

Validation check	What it protects
Required fields	Prevents blank product name, version, or install folder.
Protected install folder block	Prevents accidental writes into Windows/system locations.
Payload file existence	Prevents exporting missing files.
Duplicate payload names	Prevents install-time overwrites.
HKCU-only registry entries	Prevents unsafe machine-wide registry writes.
Shortcut/environment/association format	Prevents malformed installer actions.
Signing inputs	Prevents incomplete signing configuration.

Step 14. Build the Package

After validation succeeds, click Build Package. The authoring application saves the project and runs WIPBuild.exe with the selected project file and output folder.

```
WIPBuild.exe "C:\Path\Project.wiproj" "C:\Path\OutputFolder"
```

A successful build writes the package export files into the build output folder.

Step 15. Review the Build Output

Output item	Required result
<Product> Package.exe	The FMX setup wizard executable for the package.
install.wiproj	The packaged project manifest used by the setup wizard and

	runtime.
WipRuntime.exe	Console runtime for install, verify, repair, and uninstall automation.
payload\	Folder containing staged payload files.
build-report.txt	Inventory of the export build.
safety-report.txt	Summary of enabled safety guards.
<Product> Export.zip	ZIP archive containing the export package.

Open build-report.txt and safety-report.txt to confirm the exported package contains the expected files, actions, and safeguards.

Step 16. Test the Exported Package

Use the setup wizard or WipRuntime.exe to test the package before distribution.

```
WipRuntime.exe install.wiproj --install
```

```
WipRuntime.exe install.wiproj --verify
```

```
WipRuntime.exe install.wiproj --repair
```

```
WipRuntime.exe install.wiproj --uninstall
```

Recommended test cycle: Install, verify, repair, uninstall, and then confirm that installed files, shortcuts, registry values, file associations, environment variables, and uninstall entries are removed or restored as expected.

Required vs. Optional Build Information

Information	Required?	Reason
Project file	Required	Stores the build definition.
Build output folder	Required	Receives exported package files.
Product name	Required	Used in setup, reports, and uninstall registration.
Version	Required	Used in setup and reports.
Install folder	Required	Defines where files are installed.
Payload files	Recommended	A build can technically exist without payloads, but useful packages normally include files.

Publisher	Recommended	Improves setup and uninstall information.
Shortcuts	Optional	Creates user-accessible launch links.
Registry entries	Optional	Writes per-user HKCU values.
Environment variables	Optional	Writes current-user environment values.
File associations	Optional	Registers per-user extension handling.
Prerequisites	Optional	Documents external requirements.
Features	Optional	Documents package components.
License text	Optional	Shows license text in the setup wizard.
Signing metadata	Optional	Signs the package when configured.